

PROJEKTNA SPECIFIKACIJA

Cilj ovog projekta je da studenti, kroz izradu **jednostavne mikroservisne aplikacije**, primene **ključne DevOps prakse** i steknu iskustvo u radu sa **celokupnim procesom isporuke softvera** kakav se koristi u savremenim softverskim timovima.

Svaki student samostalno razvija **sopstveni projekat**, na kome primenjuje definisane DevOps zahteve, uz **potpunu slobodu u izboru programskog jezika, framework-a, alata i infrastrukture**, pri čemu je jedini obavezni uslov da aplikacija bude zasnovana na **mikroservisnoj arhitekturi**.

Zahtevi koje student mora da ispuni:

1. Opšti zahtevi početne aplikacije

- Svaki mikroservis mora imati bar jedan endpoint.
- Napraviti **minimalno 4 mikroservisa**.
- Dodati API Gateway.
- Mikroservisi moraju slati:
 - logove,
 - metrike i
 - *trace-ove*.
- Omogućiti različite tipove komunikacije između servisa:
 - **REST API**,
 - **Message Queue** (npr. **RabbitMQ**, **Kafka**),
 - reaktivna komunikacija (npr. **RxJava**) i
 - **gRPC** (opciono, u kombinaciji sa **REST-om**).

2. Git workflow i verzionisanje

- Korišćenje Git platforme po izboru (**GitHub**, **GitLab**, **Bitbucket...**).
- Rad isključivo kroz *feature* grane.
- Svaka značajna izmena mora ići kroz *pull request*.
- *main* grana sadrži samo stabilan kod.
- Za odbranu projekta, potrebno je:
 - priložiti link ka repozitorijumu i
 - istoriju *pull request*-ova.

3. Testiranje

- Testiranje se uvodi paralelno sa razvojem aplikacije.
- Obavezno je:
 - pisanje **unit testova** (npr. **JUnit**),
 - pisanje **end-to-end testova** i
 - svi testovi moraju prolaziti bez grešaka.
- Testovi su osnova za CI procese.

4. CI pipeline (Continuous Integration)

- Implementirati CI pipeline (npr. **GitHub Actions**, **GitLab CI**, **Jenkins**).
- Pipeline se automatski pokreće na:
 - svaki *push* i
 - svaki *pull request*.
- CI pipeline mora obuhvatati:
 - *build* aplikacije,
 - izvršavanje svih testova i
 - kreiranje build artefakata.

5. Docker kontejnerizacija

- Svaki mikroservis mora biti kontejnerizovan.
- Kreirati **Dockerfile** za bilo koju vrstu aplikacije i tehnologiju.
- Obavezno:
 - optimizacija *image*-a (npr. *multi-stage build*, gde je primenljivo),
 - korišćenje **.dockerignore** fajla,
 - konfiguracija environment promenljivih,
 - lokalni *build* i testiranje kontejnera i
 - tagovanje *image*-a.
- Objavljivanje *image*-a:
 - Opcionalno objavljivanje *image*-a na *Docker Hub*-u ili drugom *registry*-ju.
 - Ako je aplikacija u potpunosti lokalno kontejnerizovana, objavljivanje *image*-a nije obavezno.
- Docker build treba da bude integrisan u CI pipeline (automatski *build* na *push*).

6. Docker compose orkestracija

- Korišćenje Docker Compose-a za orkestraciju sistema.
- Povezati:
 - mikroservise i
 - bazu podataka (npr. **PostgreSQL** ili ekvivalent).
- Definirati:
 - mrežu između servisa,
 - environment varijable,
 - healthcheck mehanizme i
 - *volume*-e za perzistenciju baze.

7. Statička analiza koda

- Implementirati statičku analizu koda korišćenjem alata po izboru, npr:
 - **SonarCloud / SonarQube**,
 - **Lint alati (ESLint, Pylint, Checkstyle, itd.)** ili
 - drugi ekvivalentni alati.
- Statička analiza mora biti integrisana u CI pipeline.
- Analiza treba da obuhvati, gde je primenljivo:
 - *code smells*,
 - potencijalne bugove i
 - sigurnosne probleme.
- Cilj je postići zadovoljavajući kvalitet koda prema izabranom alatu.

8. Deployment

- Deploy može biti realizovan:
 - u cloud okruženju (**Render, Railway, Fly.io** ili slično),
 - ili u potpunosti lokalno (kontejnerizovano okruženje).
- Deploy obuhvata:
 - Docker aplikaciju,
 - bazu podataka,
 - konfiguraciju environment varijabli i
 - *health endpoint*.
- Ako se koristi cloud:
 - aplikacija mora biti dostupna preko javnog URL-a.

9. CD pipeline (Continuous Deployment)

- Implementirati poseban CD workflow.
- CD se automatski pokreće kada se *pull request merge*-uje u *main* granu.
- Workflow obuhvata:
 - automatski *deploy* nove verzije aplikacije,
 - korišćenje *deploy hook*-a, API-ja ili skripti (u zavisnosti od okruženja),
 - *post-deploy* proveru ispravnosti.

10. Monitoring i observability

- Implementirati monitoring i observability rešenje.
- Alat je po izboru, na primer:
 - **Grafana** (cloud ili lokalno),
 - **Prometheus + Grafana**,
 - **ELK / OpenSearch stack** ili
 - drugo slično rešenje.
- Monitoring može biti:
 - *cloud-based* ili
 - potpuno lokalni.
- Neophodno je pratiti makar:
 - *latency*,
 - *throughput* i
 - *error rate*.

Lokalno okruženje i odbrana projekta

- Studenti mogu imati kompletan sistem lokalno.
- Ceo razvojni stack mora biti kontejnerizovan.
- Projekat se demonstrira kroz:
 - Git repozitorijum,
 - CI/CD pipeline,
 - deploy (lokalni ili cloud) i
 - monitoring i observability.
- Odbrana projekta podrazumeva demonstraciju celog *DevOps workflow*-a.

Predaja projekta

Student predaje:

- Link ka Git repozitorijumu (GitHub / GitLab / Bitbucket).
- Istoriju *pull request*-ova.
- Link ka *deploy*-ovanoj aplikaciji (cloud ili lokalno).
- Projektnu dokumentaciju.

Dokumentaciju sa projektom je potrebno poslati najmanje tri dana pred termin odbrane projekta na mejl predmetnih asistenata:

- sofijadjordjevic@uns.ac.rs
- masa.saranovic@uns.ac.rs

Napomena: Odbrana projekta nosi 30 bodova.